

HW8: 完成 Quad 程序的流分析

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

`rtEqual` 的作用是判断两个 `RtValue` 是否在抽象解释的格lattice意义下相等。由于这里的数据流分析不是单纯比较整数，而是比较一种“值状态”，因此函数首先比较两者的 `ValueType` 是否一致。如果类型不同，例如一个是 `ONE_VALUE`、另一个是 `MANY_VALUES`，则直接判定不相等；如果类型相同且属于 `ONE_VALUE`，则继续比较内部保存的具体整数值；对于 `NO_VALUE` 和 `MANY_VALUES` 来说，只要类型一致即可认为等价。

`meetVal` 实现的是常量传播格上的 `meet` 运算，也就是多个控制流路径汇合时的“信息合并”逻辑。函数首先处理 `NO_VALUE`，因为它表示“尚未获得任何信息”，因此与任何值合并都应返回另一侧；随后如果任意一边已经是 `MANY_VALUES`，说明该变量不再能确定为常量，则结果直接退化为 `MANY_VALUES`；如果两边都是确定常量，则进一步比较具体值，若相等则保持该常量，否则说明不同路径上出现不同取值，于是退化为 `MANY_VALUES`。

`widenNoToMany` 的设计目的在于防止分析停留在“不知道”的状态。`NO_VALUE` 通常代表“未初始化”或“尚未分析到”，但在某些场景中继续保留这个状态会影响后续推导，因此函数将其主动扩宽（`widen`）为 `MANY_VALUES`，相当于告诉分析器：“虽然不知道具体值，但至少要认为它可能是任意值。”这种处理可以避免分析过程中由于信息缺失导致错误优化。

`mergeTemp` 用于将一个新的格值合并到临时变量表 `tv` 中。它首先检查变量是否已经存在于 `map` 中；如果不存在，则直接插入新值，并根据是否是 `NO_VALUE` 来判断分析状态是否发生变化；如果已经存在，则调用 `meetVal` 对旧值和新值做路径合并，并使用 `rtEqual` 检查合并结果是否真的发生变化。该函数不仅维护变量状态，还通过 `changed` 标记通知外层迭代算法是否需要继续迭代。

`getTerminator` 的作用是在一个基本块中找到真正的终结语句。函数从块尾向前扫描四元式列表，一旦发现 `JUMP`、`CJUMP` 或 `RETURN` 就立即返回。这种逆序扫描的写法能够保证即使块中存在一些非控制流指令，也能准确找到最后的控制流出口。

`latticeReadTemp` 实现了“带未定义检查”的变量读取逻辑。它先尝试从临时变量表中读取值；若不存在，则默认视为 `NO_VALUE`。随后，如果读到的仍然是 `NO_VALUE`，说明程序中出现了“未定义变量使用”的情况，于是函数打印警告，并将该变量强制更新为 `MANY_VALUES`。这样做的原因是：一旦变量未初始化，就无法安全地假设它是某个常量，因此必须退化为保守状态。

`evalTermLattice` 用于在格语义下计算一个 `QuadTerm` 的抽象值。函数首先判断操作数类型：如果是常量，则直接构造 `ONE_VALUE`；如果是符号名（如全局变量或函数名），由于无法静态确定其值，因此直接返回 `MANY_VALUES`；如果是临时变量，则调用 `latticeReadTemp` 获取其当前格值。整个函数的作用是把 IR 中的不同操作数统一映射到抽象解释中的 `lattice value`，为后续运算提供统一接口。

`evalRelop` 是一个关系运算求值器，用于计算两个整数之间的比较结果。函数根据字符串形式的运算

符（如 `==`、`<`、`>=`）执行对应逻辑，并返回 `optional<bool>`。使用 `optional` 的原因在于：如果遇到未知关系符，则可以返回 `nullopt` 表示“无法计算”。

`frozenConstTerm` 的作用是尝试从一个操作数中提取“冻结”的确定常量。对于立即数常量，直接返回；对于名字类型，则返回空，因为无法静态确定；对于临时变量，则检查变量表中是否存在，并且是否属于 `ONE_VALUE`。只有在分析已经确定其为唯一常量时才返回具体整数。这种“冻结”概念意味着：只有已经稳定确定的值才允许参与控制流折叠或条件化简，从而保证优化安全。

`frozenCjump` 用于在静态分析阶段尝试提前求值条件跳转。它先调用 `frozenConstTerm` 获取左右表达式的常量值，如果两边都能确定，则进一步调用 `evalRelop` 计算条件真假；否则返回 `nullopt`。这个函数的意义在于支持“条件分支折叠”：例如如果发现 `if (3 < 5)` 恒真，则编译器后续可以删除不可达分支，实现 CFG 简化。

`divOrModZeroUnsafe` 是一个安全检查函数，用来检测除法或取模运算中是否出现“除零风险”。函数只在运算符为 `/` 或 `%` 时生效，并检查右操作数是否是值为 0 的确定常量。如果满足条件，则返回 `true`。这个函数一旦发现可能除零，就不再继续常量折叠，而是退化为不确定状态，以避免编译期错误传播。

`evalBinop` 用于在 `lattice` 语义下执行二元运算。函数首先处理格状态传播：如果任意一边是 `MANY_VALUES` 或 `NO_VALUE`，则结果直接退化为 `MANY_VALUES`；随后调用 `divOrModZeroUnsafe` 检测非法运算；只有在左右两边都为确定常量时，才真正执行整数运算。对于支持的运算符（`+`、`-`、`*`、`/`、`%`），函数计算具体结果并返回 `ONE_VALUE`；对于未知运算符，则退化为 `MANY_VALUES`。也就是只有在绝对安全、绝对确定时才进行常量折叠，否则一律回退到不确定状态。

`inferEdgesRebuild` 的作用是根据当前的常量传播结果，重新推断控制流图中哪些边是真正“可执行”的。函数会遍历所有基本块，但只处理当前已经被认为可达的块；随后通过 `getTerminator` 找到块末尾的控制流语句。如果是无条件跳转 `JUMP`，则直接把对应边加入 `newE`；如果是条件跳转 `CJUMP`，则调用 `frozenCjump` 尝试静态求值条件。若条件无法确定，则说明真假分支都可能执行，于是两条边都保留；若条件可确定，则只保留真正会走到的那条边。最后函数通过比较旧边集与新边集是否不同，决定是否更新，并利用 `changedOut` 通知外层分析继续迭代。

`mapsEqualRt` 用于比较两个变量状态表是否完全一致。由于这里的 `map` 存储的是 `lattice value`，而不是普通整数，因此不能直接用 `==` 判断，而是需要逐项调用 `rtEqual`。函数首先检查 `map` 大小是否一致，然后遍历第一个 `map`，检查第二个 `map` 中是否存在同样键值，并比较对应 `RtValue` 是否等价。这个函数其实就是在判断一次传播之后变量状态是否已经稳定。如果没有任何变化，说明分析已经收敛，可以停止迭代。

`propagateReachability` 的作用是根据当前有效的 CFG 边集合，计算哪些基本块是“可达”的。函数首先将入口块标记为可达，然后不断遍历边集合：如果某条边的起点已经可达，而终点尚不可达，则把终点也标记为可达，并继续迭代。经过常量传播和条件折叠之后，一些原本存在于 CFG 中的边已经不再可能执行，因此很多块会变成不可达块，而这些块后续就可以被删除或忽略。

`interpretBlock` 负责“执行”一个基本块中的所有 IR 指令，但这里的执行并不是运行真实程序，而是在 `lattice` 上进行抽象求值。函数首先获取当前块编号，然后遍历块内所有四元式，根据不同指令类型分别处理。对于 `PHI` 节点，它会检查哪些前驱边当前仍然可执行，仅合并这些路径上传来的变量值，并通过 `meetVal` 得到最终 `PHI` 结果；对于普通赋值 `MOVE`，则调用 `evalTermLattice` 计算右值并更新目标变量；对于二元运算 `MOVE_BINOP`，则先分别求左右操作数的抽象值，再调用 `evalBinop` 做常量传播。对于内存读取 `LOAD`、函数调用 `MOVE_CALL` / `MOVE_EXTCALL` 以及指针运算 `PTR_CALC`，由于分析无法静

态确定其结果，因此统一退化为 `MANY_VALUES`。

`buildLabel2Block` 负责建立从 label 编号到基本块对象的映射表。函数遍历函数中的所有基本块，并将 `entry_label->num` 作为 key，对应块指针作为 value 存入 `label2block`。这样做的目的，是为了后续分析中能够通过 label 快速定位 CFG 中的目标块，而不需要每次都线性遍历整个 block 列表。这个映射结构在跳转解析、CFG 构建、可达性分析以及优化阶段都会频繁使用，相当于控制流分析中的索引表。

`Opt::calculateBT` 负责不断迭代控制流与数据流信息，直到达到全局不动点。函数开始时先建立 label 到 block 的映射，并清空变量格值表与块可执行状态表；随后把入口块标记为可执行。整个分析被放在一个 `while (globalChanged)` 循环中，只要 CFG 或变量状态还有变化，就继续分析。循环内部首先调用 `inferEdgesRebuild` 根据当前常量信息重建“真正可能执行”的边，然后利用 `propagateReachability` 重新计算哪些块仍然可达。之后函数更新 `block_executable` 集合，动态删除不可达块。接下来，它又进入一个局部迭代，对所有当前可执行块调用 `interpretBlock`，在 lattice 上解释执行程序，并不断传播新的变量信息，直到 `mapsEqualRt` 判断变量状态稳定。最后再把分析结果写回 `temp_value`。它实际上就是 SCCP 的标准结构：控制流剪枝与常量传播交替迭代，互相增强，直到 CFG 和 lattice 全部稳定。

`peekTempLattice` 用于从优化器当前的 `temp_value` 表中读取某个临时变量的 lattice 值。如果变量已经存在，则返回对应值；否则返回默认构造的 `RtValue`，一般是 `NO_VALUE`。它是为了给后续 IR 替换与常量折叠提供统一的查询接口，而不直接暴露内部 map 细节。

`substTerm` 的作用是在优化阶段尝试把一个操作数替换成真正的常量。函数首先处理立即数与名字类型，它们本身已经是最终形式，因此直接 clone；如果是 TEMP，则会查询该临时变量在 lattice 中的状态。如果分析已经确定其为 `ONE_VALUE`，则直接构造新的常量 `QuadTerm` 替换它；否则保留原 TEMP。这个函数把原来的寄存器引用直接改写成字面值。

`singleTempUse` 会创建一个新的 `set<Temp*>`，并把给定 temp 插入进去后返回。这个函数可以用于构造 use 集，方便后续数据流分析统一处理。`singleTempDef` 创建的则是定义集合。函数同样分配一个新的 `set<Temp*>`，插入指定变量并返回。这个函数可以用于活跃变量分析、DU 链构建或者 SSA 分析等需要记录变量定义位置的场景。

`collectTermUses` 的作用是统计一个操作数中的 TEMP 使用次数。函数只关注 TEMP 类型，因为常量和名字不会影响局部变量的数据流统计；如果操作数确实是 TEMP，则将对应变量的编号在 `usec` 中计数加一。

`countStmtUses` 负责统计一条 IR 指令中所有被“读取”的 TEMP。函数根据不同 `QuadKind` 分类处理：例如 `MOVE` 会统计源操作数，`STORE` 会统计地址与写入值，`MOVE_BINOP` 会统计左右操作数，而调用语句则会遍历所有参数。对于 `PHI` 节点，它会统计所有输入变量；对于 `CJUMP`，则统计条件比较使用的变量。该函数把 IR 中“哪些变量被使用了”这一信息显式收集起来，后续优化便可以依据 use-count 判断某个定义是否已经无用。

`getDefTemp` 的作用是提取“一条语句定义了哪个 TEMP”。不同 IR 指令定义 TEMP 的位置不同，例如 `MOVE` 定义 `dst`，`LOAD` 定义加载结果，`MOVE_BINOP` 定义运算结果，而 `PHI` 定义合并后的 SSA 变量。函数通过区分不同 `QuadKind`，把这些定义位置统一抽象成一个接口，并返回 TEMP 编号。如果某条语句并不定义变量，则返回 -1。

`storeMightBeObservable` 用于判断某条语句是否具有“可观察副作用”。函数认为 `STORE`、普通 `CALL` 与外部调用 `EXTCALL` 都可能影响程序外部状态，因此不能轻易删除。只有纯计算且无副作用的代码才

能安全消除。比如即使某个函数返回值没人使用，只要它可能修改内存、打印内容或访问 IO，就不能删掉。这个函数会在死代码消除阶段作为安全检查条件。

`moveBinopIsDivLike` 用于判断某个二元运算是否属于“除法类操作”，即 `/` 或 `%`。单独封装这一判断，是因为除法和取模具有特殊风险，例如除零异常，因此后续优化中需要特殊处理。

`moveBinopStmtIsDivLike` 在语句层面判断某条 `MOVE_BINOP` 是否是除法或取模语句。它先检查语句种类是否为 `MOVE_BINOP`，然后调用 `moveBinopIsDivLike` 判断具体操作符。

`recomputeExitLabels` 的作用是在 CFG 被修改后，重新计算一个基本块的出口标签集合。函数首先获取块的终结指令，并确保 `exit_labels` 容器存在；随后根据终结语句类型更新出口信息：对于 `JUMP`，只有一个目标标签；对于 `CJUMP`，则同时记录真假两个分支。这个函数的重要性在于：经过常量传播、条件折叠或 CFG 剪枝后，块的控制流结构可能已经变化，因此必须重新同步 `exit_labels`，否则 CFG 会与真实 IR 不一致。

`transformStatementList` 会根据之前 SCCP 分析得到的 `lattice` 信息与可执行边信息，对一个基本块中的语句列表进行重构。函数首先创建新的输出语句列表 `out`，然后逐条扫描原始 IR。对于 `PHI` 节点，它会根据当前仍可执行的 CFG 边过滤掉已经不可达的输入；如果过滤后没有任何输入，则整个 `PHI` 可以删除；如果只剩一个输入，则说明该 SSA 合并已经退化成普通赋值，于是直接转换为 `MOVE`，并进一步尝试常量化；否则保留精简后的 `PHI`。对于普通语句，函数会调用 `substTerm` 将 `TEMP` 替换为已知常量，例如 `MOVE`、`LOAD`、`STORE`、`RETURN` 等都会把其内部操作数做常量替换。对于 `MOVE_BINOP`，如果分析已经确定结果是确定常量，则直接把整条运算替换成常量赋值，从而实现真正的常量折叠。对于 `CJUMP`，函数会尝试静态求值条件：若条件恒真或恒假，则直接替换成无条件 `JUMP`，从而删除死分支；否则仅对子表达式做常量替换后保留条件跳转。调用语句、指针运算和外部调用则会递归替换其参数中的 `TEMP`。

`foldSequentialConstTempsInBlocks` 的作用是进一步执行一种局部的顺序常量传播优化。函数遍历每个基本块，并维护一个 `binding` 表，用于记录“某个 `TEMP` 当前绑定到哪个常量”。当遇到 `MOVE t = CONST` 时，就把该 `TEMP` 与对应常量记录下来；之后如果在同一块中发现 `RETURN t`，并且该 `TEMP` 在 `binding` 中存在，则直接把返回值替换成字面常量。

`findBlockWithEntry` 会在线性遍历函数所有基本块后，返回入口 `label` 编号等于 `entryNum` 的块。如果找不到，则返回 `nullptr`。这个函数本质上是 `label` 到 `block` 的查找工具，通过 `label` 快速定位目标块。

`blockDefinesTemp` 的作用是判断某个基本块是否定义了指定 `TEMP`。函数会遍历块内所有语句，并调用 `getDefTemp` 提取每条语句定义的 `TEMP` 编号；只要发现与目标编号一致，就立即返回 `true`，否则最终返回 `false`。这个函数实际上是在做局部 `def` 集查询。

`insertBeforeTerminator` 的作用是在一个基本块的“终结语句之前”插入新的 IR。函数先获取块的语句列表；如果块为空，则直接插入；否则检查最后一条语句是否属于控制流终结指令 `JUMP`、`CJUMP` 或 `RETURN`。如果是，则把新语句插入到倒数第二位置，保证控制流终结语句仍然位于块尾；否则直接追加到末尾。CFG 中的 `terminator` 必须保持在基本块最后，否则控制流结构会被破坏。因此任何中途插入的新指令，都必须显式避开 `terminator`。

`latticeLookup` 用于从优化器当前的 `temp_value` 表中读取某个 `TEMP` 的抽象值。函数首先在 `map` 中查找对应编号；如果不存在，则返回默认构造的 `RtValue`，通常意味着 `NO_VALUE`；否则直接返回保存的 `lattice` 状态。这个函数与前面的 `peekTempLattice` 类似，但它用来快速判断某个变量当前是否已经被分析为确定常量。

`existsPhiIncomingConstOutsidePredDefs` 的作用是检测程序中是否存在一种特殊情况：某个 PHI 节点接收到的输入变量，在其对应前驱块中并没有被重新定义，但该变量在全局 `lattice` 中已经被推导成了确定常量。函数会遍历所有基本块，并检查块开头的 PHI 节点；对于每个 PHI 输入 (`Temp`, `Pred`)，它会先找到对应前驱块，再检查该 `TEMP` 是否在前驱块内被定义。如果没有定义，但 `lattice` 却表明它是 `ONE_VALUE`，函数立即返回 `true`。这意味着“常量值来自支配路径或更早位置，而不是来自前驱块本身”。

`bubbleSortPureConstMovesByDst` 的作用是对“纯常量赋值语句”按照目标 `TEMP` 编号进行排序。函数会遍历每个基本块，并不断执行冒泡排序：只有当相邻两条语句都是 `MOVE CONST` 时，才会比较它们的目标 `TEMP` 编号；如果前者编号更大，则交换位置。这个函数本身并不会改变程序语义，因为纯常量赋值之间不存在依赖关系，因此顺序可交换。

`normalizePhiConstantSources` 的目标是“显式化” PHI 节点中的常量来源。函数首先调用 `existsPhiIncomingConstOutsidePredDefs` 检测是否真的需要进行这种修复；如果不存在问题，则直接退出。否则，它会遍历所有 PHI 输入，对于那些 `lattice` 已确定为常量、但在对应前驱块中没有显式定义的 `TEMP`，函数会为其分配一个新的 `TEMP` 编号，并尝试把该常量真正放到前驱块中。如果前驱块本来就存在 `MOVE CONST` 定义，则直接改写这条 `MOVE` 的目标 `TEMP`；否则就在前驱块终结语句之前插入新的 `MOVE temp = const`。最后再把 PHI 输入改成新的 `TEMP`。实际上是在“把隐式常量传播重新物化为显式 SSA 定义”，从而保证 PHI 的每个输入都真正来自对应前驱块。

`deadCodeSweep` 实现的是经典的死代码消除。函数采用迭代方式不断删除无用语句，直到再也删不动为止。每轮开始时，它会统计整个函数中所有 `TEMP` 的使用次数 `usec`；随后重新扫描所有语句。如果一条语句定义了某个 `TEMP`，但这个 `TEMP` 的 `use-count` 为 0，并且该语句没有可观察副作用（由 `storeMightBeObservable` 判断），那么该语句就可能是死代码。对于简单常量赋值 `MOVE CONST`、PHI 节点，以及非除法类 `MOVE_BINOP`，函数会直接删除它们。这里特别排除了除法与取模，因为即使结果没被使用，它们也可能触发除零异常，因此仍具有语义影响。每次删除后，函数都会重新迭代，因为删掉一条语句后，其他语句也可能变成新的死代码。

`Opt::modifyFunc` 负责把之前 SCCP 分析得到的信息应用到代码中。函数首先重新推导一次有效边集合；然后根据 `block_executable` 删除所有不可达基本块，实现 CFG 剪枝。接下来，对保留下来的每个块调用 `transformStatementList`，执行常量替换、条件折叠、PHI 精简等实际 IR 改写。随后函数检查是否存在“PHI 常量来源不规范”的情况，如果存在，则调用 `normalizePhiConstantSources` 修复 SSA。之后又执行局部常量折叠 `foldSequentialConstTempsInBlocks` 与死代码删除 `deadCodeSweep`。最后，函数会额外增加 `last_temp_num`，并在需要时调用 `bubbleSortPureConstMovesByDst` 对常量赋值排序，同时重新计算所有块的出口标签。

`Opt::optFunc` 是单个函数优化流程的统一入口。函数首先构建 `label` 到 `block` 的映射，然后调用 `calculateBT` 执行 SCCP 分析与 `lattice` 求解，之后调用 `modifyFunc` 根据分析结果真正改写 IR，最后返回优化后的函数对象。整个函数实际上对应“优化 pipeline”中的一个完整 pass：分析 → 求不动点 → 修改 IR。

`optProg` 则是程序级别的优化入口。函数会创建一个新的 `QuadProgram`，保留原程序的全局 `label` 与 `TEMP` 编号信息，然后遍历程序中的每个函数，为每个函数创建一个 `Opt` 对象并调用 `optFunc` 进行优化，最后把优化后的函数加入新程序中返回。这个函数本质上实现了“对整个 IR 程序逐函数执行 SCCP 优化”的过程，是整个优化器的最顶层驱动入口。

开发过程和测试结果

开发过程如下：

Graph	Description	Date	Author	Commit
	feat: 完成了HW8的全部内容并且得到正确输出 Merge branch "master" of gitee.com:fudanCompiler/fducompilerh2026	7 May 2026 14:28	23300240034-chen...	b0c3adb0
	HW8 test output (3,5,6) changed Remove useless libsysy	7 May 2026 14:25	23300240034-chen...	d7331d98
	HW8 test output (3,5,6) changed HW8: README tests	3 May 2026 23:06	xywang2772	5176bf3b
	HW8 test output (3,5,6) changed HW8 test files added	1 May 2026 18:38	王晓阳	35f39d94
	HW8 test output (3,5,6) changed HW8 README.md	30 Apr 2026 23:24	王晓阳	4cd6e47a
	HW8 test output (3,5,6) changed HW8 README added	30 Apr 2026 23:02	王晓阳	a10a8658
	HW8 test output (3,5,6) changed feat: 完成了SSA变换的所有逻辑并校正tests输出	30 Apr 2026 23:01	王晓阳	5b346ccd
	HW8 test output (3,5,6) changed begin: 添加了HW7所有文件	30 Apr 2026 21:51	王晓阳	0f8b2379
	HW8 test output (3,5,6) changed HW8 added	25 Apr 2026 14:37	23300240034-chen...	1fcd638a
	HW8 test output (3,5,6) changed HW8 added	25 Apr 2026 14:31	23300240034-chen...	e9820f89
	HW8 test output (3,5,6) changed HW8 added	23 Apr 2026 17:13	王晓阳	dce8e175

make run 之后得到的所有测试的结果，与老师给出的参考测试结果完全一致。并且在终端得到的输出片段如下：

```
cd .. > /dev/null 2>&1
Reading opttest1.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest1.4-ssa-xml.quad
Writing optimized Quad to file: opttest1.4-ssa-opt.quad
-----Done---
Reading opttest2.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest2.4-ssa-xml.quad
Writing optimized Quad to file: opttest2.4-ssa-opt.quad
-----Done---
Reading opttest3.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest3.4-ssa-xml.quad
Writing optimized Quad to file: opttest3.4-ssa-opt.quad
-----Done---
Reading opttest4.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest4.4-ssa-xml.quad
Writing optimized Quad to file: opttest4.4-ssa-opt.quad
-----Done---
Reading opttest5.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest5.4-ssa-xml.quad
Warning: undefined use of temp t100, treating as MANY_VALUES
Writing optimized Quad to file: opttest5.4-ssa-opt.quad
-----Done---
Reading opttest6.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest6.4-ssa-xml.quad
Warning: undefined use of temp t100, treating as MANY_VALUES
Writing optimized Quad to file: opttest6.4-ssa-opt.quad
-----Done---
Reading opttest7.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest7.4-ssa-xml.quad
Warning: undefined use of temp t100, treating as MANY_VALUES
Writing optimized Quad to file: opttest7.4-ssa-opt.quad
-----Done---
Reading opttest8.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest8.4-ssa-xml.quad
Writing optimized Quad to file: opttest8.4-ssa-opt.quad
-----Done---
Reading opttest9.4-ssa-xml.quad
Reading Quad (SSA) from xml: opttest9.4-ssa-xml.quad
Writing optimized Quad to file: opttest9.4-ssa-opt.quad
-----Done---
root@cyx:~/fducompilerh2026/fducompilerh2026/HW8#
```

参考资料

- 虎书中文版，“静态单赋值形式”，“数据流分析/格与不动点”，“循环优化”部分
- 大模型：DeepSeek/Cursor，协助理解虎书内容和修改代码中遇到的临时变量沿用和变量0何时处理问题。